

Bangladesh University of Engineering and Technology

Department of Computer Science and Engineering

CSE 210
Computer Architecture Sessional
Assignment on 4-bit MIPS Design and Simulation

Section: C2 **Group No.:** 6

Group Members:

Md. Fuad Al Alam	(2205167)
Abdullah Siham Rahman	(2205169)
Ali Akib Al-Amin	(2205170)
Fahim Faiaz Adib	(2205177)
Gazi Faiaz Zafor Niloy	(2205179)
Avrajit Biswas	(2105126)

Date of Submission: March 28, 2026

Contents

1 Introduction **2**

2 Instruction Set **3**
2.1 Design Specification 3
2.2 Register Encoding 4
2.3 Opcode Assignment – Section C2, Group 6 4
2.4 ALU Operation Encoding 5

3 Circuit Diagram **5**
3.1 Overall Datapath 6
3.2 Instruction Memory 6
3.3 Register File 7
3.4 ALU 7
3.5 Control Unit 8
3.6 Data Memory 9

4 How to Write and Execute a Program **9**
4.1 Writing Assembly Code 9
4.2 Automatic Machine Code Generation (Python Assembler) 9
4.3 Simulation in Logisim 10

5 Special Features Implemented **11**
5.1 Stack Pointer and Push/Pop Pseudo-Instructions 11

6 Discussion **11**

1 Introduction

MIPS (Microprocessor without Interlocked Pipelined Stages) is a RISC (Reduced Instruction Set Computer) ISA (Instruction Set Architecture). It was originally developed in 1985 by MIPS Computer Systems. The hallmark of MIPS is its regularity: every instruction follows a fixed, rigid format, making hardware decode straightforward. For example, arithmetic operations always operate on registers:

Operation	**Instruction**	**Action**
Addition	`add $t1, $t2, $t3`	`$t1 = $t2 + $t3`
Subtraction	`sub $t1, $t2, $t3`	`$t1 = $t2 - $t3`

In this assignment, we design and simulate (in software) a *modified and reduced* version of the MIPS instruction set tailored to a 4-bit datapath. The key design constraints are:

- 8-bit address bus, 4-bit data bus.
- 4-bit ALU and 4-bit registers.
- Seven registers: `$zero, $t0, $t1, $t2, $t3, $t4, $sp`.
- 16-bit instruction word with four formats (R, S, I, J).
- Microprogrammed control unit using a ROM-based control word store.
- Automatic assembly-to-machine-code conversion via a Python assembler.

A *datapath* is the collection of functional units (ALU, registers, MUXes, memories) that process data and addresses in the CPU. Figure 1 shows the classical single-cycle MIPS datapath with a control unit, which served as the conceptual starting point for our reduced design.

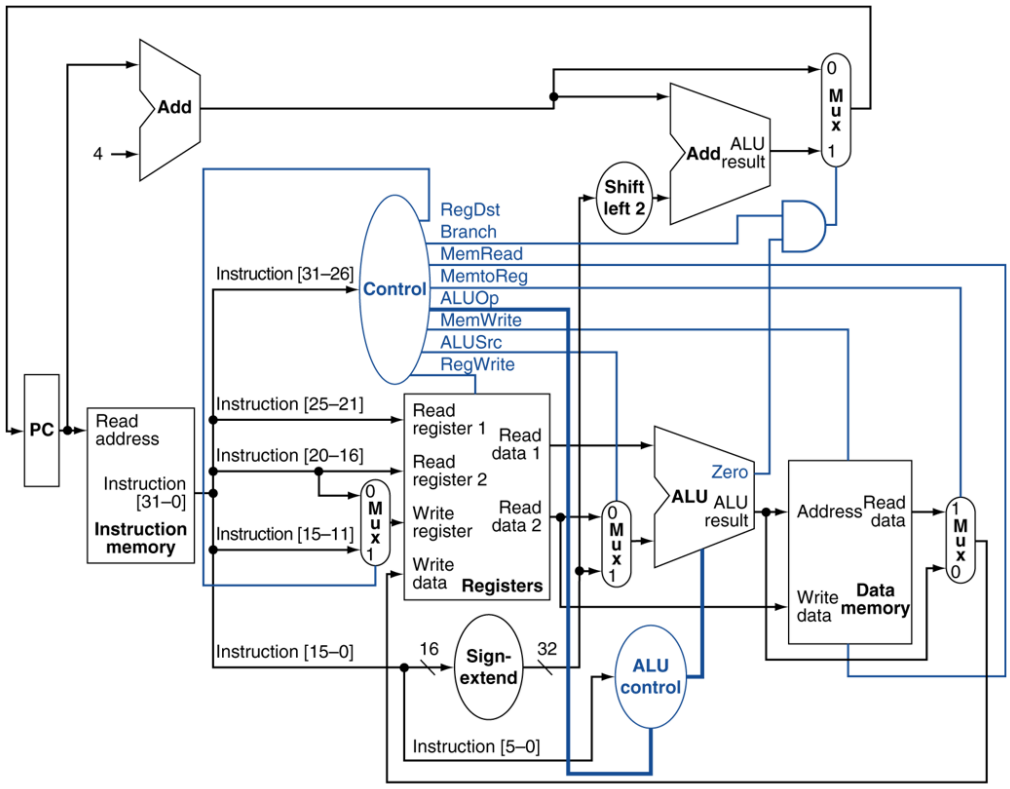


Figure 1: Classical single-cycle MIPS datapath with control.

2 Instruction Set

2.1 Design Specification

Our reduced MIPS instructions are **16 bits** long. The four instruction formats are defined as follows.

R-type (Register)

Opcode	Src Reg 1	Src Reg 2	Dst Reg
4 bits	4 bits	4 bits	4 bits

Used for arithmetic/logic operations that read two source registers and write to a destination register.

S-type (Shift)

Opcode	Src Reg 1	Dst Reg	Shamt
4 bits	4 bits	4 bits	4 bits

Used for shift operations (`sll`, `srl`). The *Shamt* field encodes the number of bit positions to shift.

I-type (Immediate)

Opcode	Src Reg 1	Src Reg 2 / Dst Reg	Addr. / Immdt.
4 bits	4 bits	4 bits	4 bits

Used for immediate-operand arithmetic, memory access, and branch instructions.

J-type (Jump)

Opcode	Target Jump Address	0 (padding)
4 bits	8 bits	4 bits

Used for the unconditional jump instruction (`j`).

2.2 Register Encoding

The six registers and their 4-bit encodings used throughout instruction fields are:

Register	Description	4-bit Code
`$zero`	Hardwired zero	`0000`
`$t1`	Temporary 1	`0001`
`$t2`	Temporary 2	`0010`
`$t3`	Temporary 3	`0011`
`$t4`	Temporary 4	`0100`
`$t0`	Temporary 0	`0101`
`$sp`	Stack Pointer	`0111`

Note: `$t0` is encoded as `0101` (not `0001`) as defined in the assembler.

2.3 Opcode Assignment – Section C2, Group 6

The opcode sequence assigned to our group (Section C2, Group 6) is:

`J D B L E M I A H P N K G O C F`

Each position in the sequence corresponds to the opcode value (0–15):

Opcode	ID	Instruction	Type	Format
0	J	`srl`	Logic	S-type
1	D	`subi`	Arithmetic	I-type
2	B	`addi`	Arithmetic	I-type
3	L	`lw`	Memory	I-type
4	E	`and`	Logic	R-type
5	M	`sw`	Memory	I-type
6	I	`sll`	Logic	S-type
7	A	`add`	Arithmetic	R-type
8	H	`ori`	Logic	I-type
9	P	`j`	Control	J-type
10	N	`beq`	Control	I-type
11	K	`nor`	Logic	R-type
12	G	`or`	Logic	R-type
13	O	`bneq`	Control	I-type
14	C	`sub`	Arithmetic	R-type
15	F	`andi`	Logic	I-type

2.4 ALU Operation Encoding

The 3-bit `ALUOp` field encodes the operation performed by the ALU:

ALUOp (3-bit)	Operation	Description
`000`	`and`	Bitwise AND
`001`	`or`	Bitwise OR
`010`	`add`	Addition
`011`	`sub`	Subtraction
`100`	`nor`	Bitwise NOR
`101`	`sll`	Logical shift left
`110`	`srl`	Logical shift right

3 Circuit Diagram

This section presents the circuit diagrams for each major component of our 4-bit MIPS design, implemented and simulated in Logisim Evolution (v4.1.0).

3.1 Overall Datapath

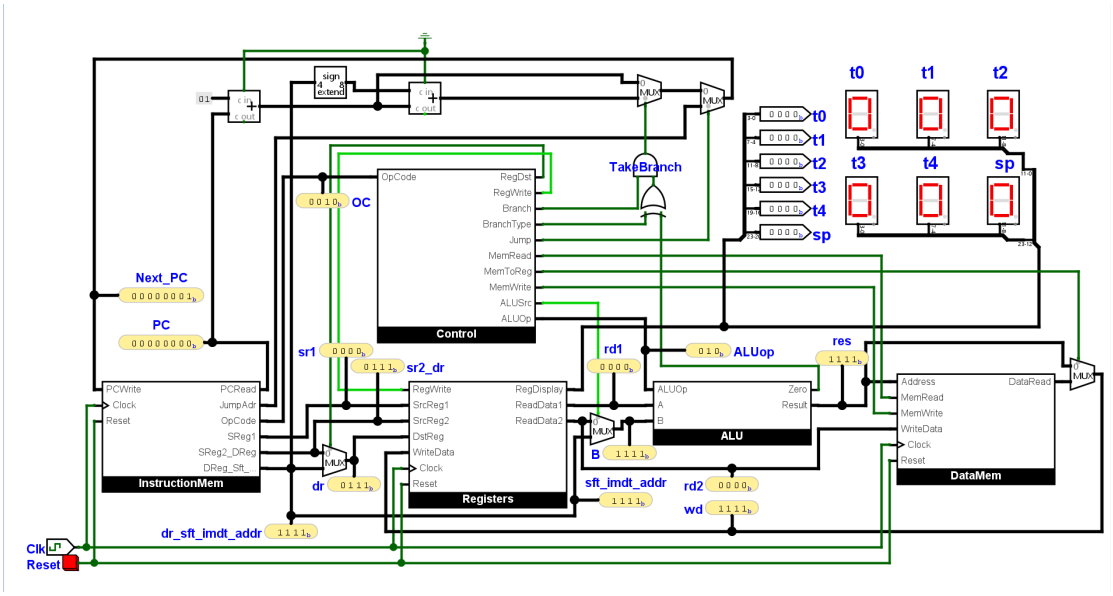


Figure 2: Overall datapath for the 4-bit MIPS processor.

The datapath integrates the instruction memory (ROM), register file, ALU, data memory (RAM), and all interconnecting MUXes and control signals into a single-cycle design.

3.2 Instruction Memory

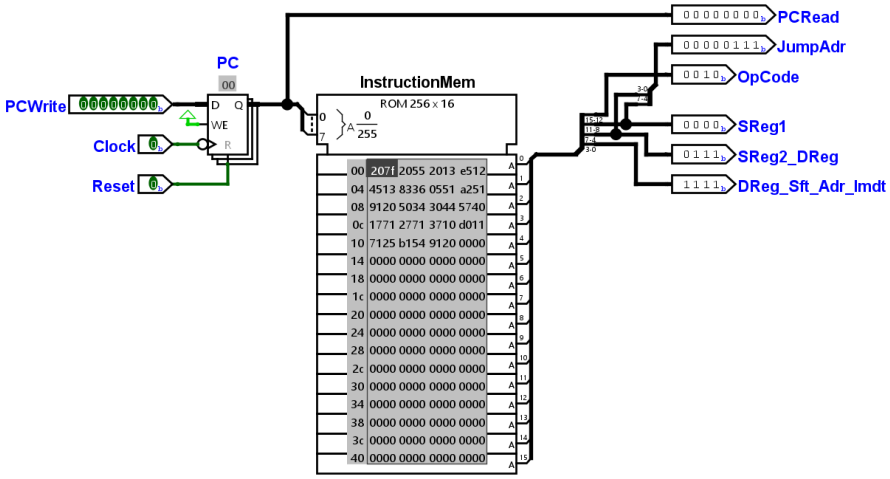


Figure 3: Instruction memory (256-byte ROM) and PC block.

The instruction memory is implemented as a 256-byte ROM addressed by an 8-bit Program Counter (PC). The 16-bit instruction word is split into Opcode (bits 15–12), fields 11–8, 7–4, 3–0, and the 8-bit jump address (bits 11–4) for J-type instructions.

3.3 Register File

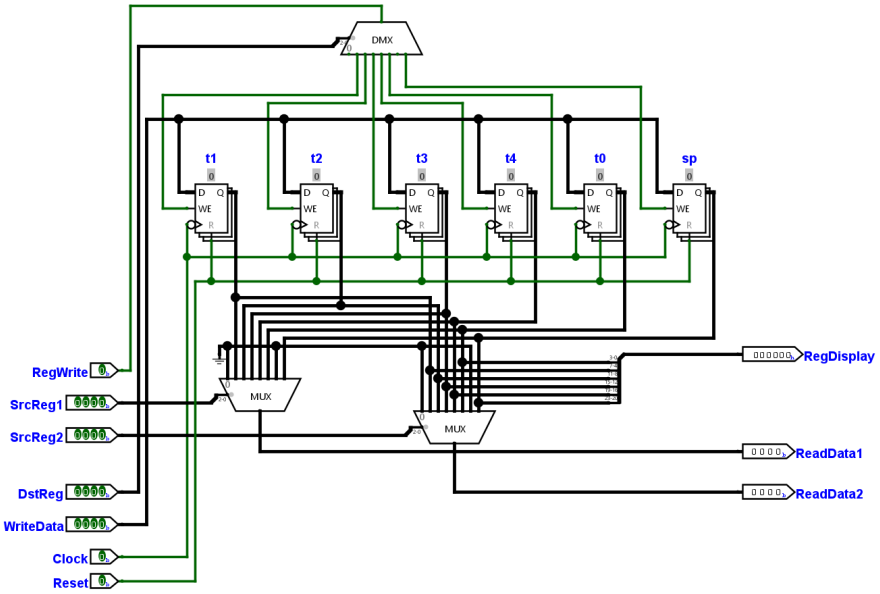


Figure 4: 4-bit register file (`$zero`, `$t0`–`$t4`).

The register file contains six 4-bit registers. `$zero` is hard-wired to 0. A decoder (DMX) routes the write data to the correct destination register, while two MUXes select the read data for the two source operands.

3.4 ALU

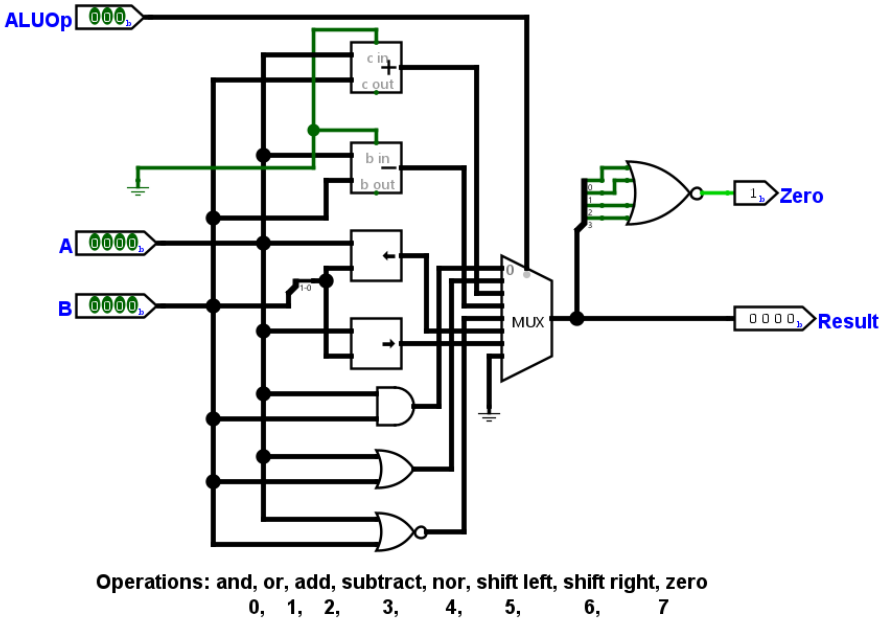


Figure 5: 4-bit ALU supporting add, sub, and, or, nor, sll, srl.

The ALU accepts two 4-bit operands and a 3-bit ALUOp control signal. It supports the following operations: addition, subtraction, bitwise AND, bitwise OR, bitwise NOR, logical shift left, and logical shift right. A Zero flag is generated by NOR-ing all output bits, used for branch evaluation.

3.5 Control Unit

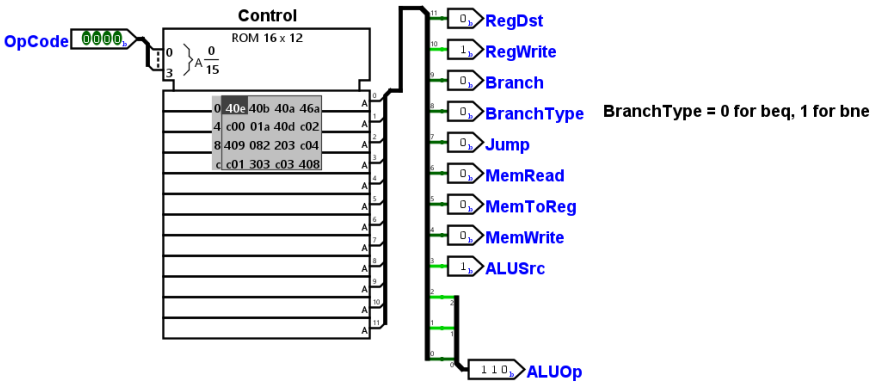


Figure 6: Microprogrammed control unit (16-word ROM).

The control unit is *microprogrammed*: a 16-word ROM is addressed by the 4-bit opcode. Each ROM word encodes a *control word* containing 12 control signals: RegDst, RegWrite, Branch, BranchType, Jump, MemRead, MemToReg, MemWrite, ALUSrc, and ALUOp (3 bits).

The control signals for each instruction are summarised in Table 1. The 12-bit control word is stored in the ROM in the order: `RegDst | RegWrite | Branch | BranchType | Jump | MemRead | MemToReg | MemWrite | ALUSrc | ALUOp[2:0]`.

Table 1: Control signals for each instruction.

Instr.	RegDst	RegWr	Branch	BrType	Jump	MemRd	Mem2Reg	MemWr	ALUSrc	ALUOp
`add`	1	1	0	0	0	0	0	0	0	010
`sub`	1	1	0	0	0	0	0	0	0	011
`and`	1	1	0	0	0	0	0	0	0	000
`or`	1	1	0	0	0	0	0	0	0	001
`nor`	1	1	0	0	0	0	0	0	0	100
`sll`	0	1	0	0	0	0	0	0	1	101
`srl`	0	1	0	0	0	0	0	0	1	110
`addi`	0	1	0	0	0	0	0	0	1	010
`subi`	0	1	0	0	0	0	0	0	1	011
`andi`	0	1	0	0	0	0	0	0	1	000
`ori`	0	1	0	0	0	0	0	0	1	001
`lw`	0	1	0	0	0	1	1	0	1	010
`sw`	X	0	0	0	0	0	0	1	1	010
`beq`	X	0	1	0	0	0	0	0	0	011
`bneq`	X	0	1	1	0	0	0	0	0	011
`j`	X	0	0	0	1	0	0	0	0	010

RegDst: 0 = bits[7:4] (I/S-type dest), 1 = bits[3:0] (R-type dest). **BrType:** 0 = `beq`, 1 = `bneq`. **ALUOp** encoding: see Section 2.4.

3.6 Data Memory

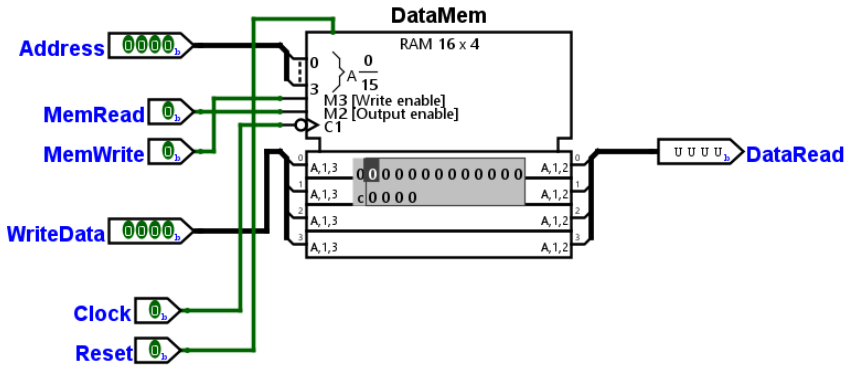


Figure 7: Data memory (16-nibble RAM) for `lw`/`sw`.

4 How to Write and Execute a Program

4.1 Writing Assembly Code

Assembly programs are written using the six available registers (`$zero`, `$t0`–`$t4`) and the sixteen instructions listed in Section 2. Comments begin with `//`. Below is a sample program that computes $n = a + b - c$ (assuming a, b, c are pre-loaded into `$t1`, `$t2`, `$t3` respectively):

```
// Compute n = a + b - c
// a in $t1, b in $t2, c in $t3, result in $t0
add  $t0, $t1, $t2     // $t0 = a + b
sub  $t0, $t0, $t3     // $t0 = (a + b) - c
```

Listing 1: Sample assembly program

4.2 Automatic Machine Code Generation (Python Assembler)

An assembler written in Python converts `.asm` source files into `.hex` and `.bin` machine code files automatically.

Encoding rules

- Registers are encoded as 4-bit values (see Section 2.2).
- Opcodes are as specified in Section 2.3.
- For R-type: `[opcode(4)][rs(4)][rt(4)][rd(4)]`.
- For S-type: `[opcode(4)][rs(4)][rd(4)][shamt(4)]`.
- For I-type: `[opcode(4)][rs(4)][rt/rd(4)][imm(4)]`.
- For J-type: `[opcode(4)][addr(8)][0000]`.
- Branch offsets (`beq`, `bneq`) are PC-relative and encoded as a 4-bit two's complement value.
- `sw` and `lw` use the format `[opcode(4)][base_reg(4)][dst/src_reg(4)][offset(4)]`.

Stack pointer support

The assembler includes a built-in stack pointer (`$sp`, encoded as `0111`). When `SP_ENABLED` is set to `True`, the assembler automatically initialises `$sp` to 15 and expands the pseudo-instructions `push` and `pop` into the corresponding `sw`/`lw` and `subi`/`addi` sequences.

Assembler usage

```
python C2_G6_MIPS_to_BIN.py
# Reads:   input.asm
# Outputs: output.hex  (for loading into Logisim ROM)
#          output.bin  (binary representation with annotations)
```

Listing 2: Running the assembler

The core encoding logic is structured as follows:

```
mapping = {
    'A':'add','B':'addi','C':'sub','D':'subi',
    'E':'and','F':'andi','G':'or', 'H':'ori',
    'I':'sll','J':'srl','K':'nor','L':'lw',
    'M':'sw', 'N':'beq','O':'bneq','P':'j'
}

sequence = 'JDBLEMIAHPNKGOCF'   # Group C2-6 opcode order

# Register encoding (note: $t0 = 0101, not 0001)
reg_table = {
    '$zero':'0000', '$t1':'0001', '$t2':'0010',
    '$t3' :'0011',  '$t4':'0100', '$t0':'0101',
    '$sp' :'0111'
}

ALU_OP = {
    'and':'000','or':'001','add':'010','sub':'011',
    'nor':'100','sll':'101','srl':'110'
}
```

Listing 3: Assembler snippet (Python) – opcode and register tables

4.3 Simulation in Logisim

1. Write the assembly program in `input.asm`.
2. Run `python C2_G6_MIPS_to_BIN.py` to produce `output.hex` and `output.bin`.
3. Open `MIPS_C2_G6_HSC22.circ` in Logisim Evolution (v4.1.0).
4. Right-click the Instruction Memory ROM → *Edit Contents* → paste the contents of `output.hex`.
5. Left-click the *clock* at bottom left corner of main once then twice (or press `Ctrl+T` two times serially) to step through the program one cycle. Keep doing this for all instructions to run the program.
6. Observe register and memory values update each clock cycle.

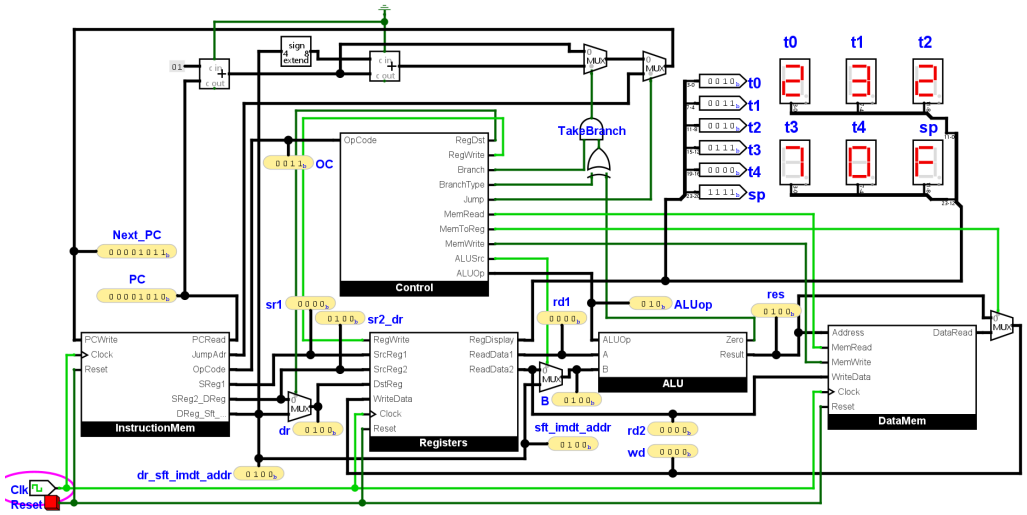


Figure 8: Logisim simulation in progress showing register and memory state.

5 Special Features Implemented

5.1 Stack Pointer and Push/Pop Pseudo-Instructions

When `SP_ENABLED = True` the assembler prepends an `addi $sp, $zero, 15` initialisation instruction and expands the following pseudo-instructions automatically:

Pseudo-instruction	Expansion
`push $tx`	`sw $tx, 0($sp)` → `subi $sp, $sp, 1`
`pop $tx`	`addi $sp, $sp, 1` → `lw $tx, 0($sp)`

6 Discussion

In this assignment, we designed and simulated a modified, reduced version of the MIPS instruction set architecture on a 4-bit datapath. Our implementation deviates from original MIPS in several ways: instructions were reduced to 16 bits instead of 32, special instruction type S-format was introduced to allow logical shifts and the R-format was heavily modified to fit into 16 bits (opcode encodes the operation instead of funct as seen in standard MIPS). Additionally, our design makes us increment Program Counter (PC) by 1 each time instead of 4. And the most noticeable part is the low amount of memory and registers, which are all temporary registers only. However the functionality is mostly intact and it can simulate most hypothetical MIPS usage scenarios, provided that the numbers are always integers between -8 and 7 and the small data and instruction memories are sufficient. Ultimately it was a great learning experience for us and we learnt a lot about how actual CPUs of computers run Alhamdulillah.